Lab 1: The Multi-Armed Bandit Problem and MDP Foundations

Morgan Cooper
MSDS 684 — Reinforcement Learning

March 16, 2026

Section 1: Project Overview

This lab focused on the multi-armed bandit problem. This problem is simple, yet foundational to understanding reinforcement learning (RL). The multi-armed bandit problem consists of a bandit (representative of a slot machine) with $k = 10$ arms, where each arm returns some hidden reward value. An agent selects an arm, receives a reward, and in response attempts to "learn" which arm will yield the highest reward over time.

The multi-armed bandit problem primarily exposes the exploration-exploitation tradeoff. An agent who always exploits (chooses the most optimal known action) will potentially miss out on discovering the true optimal action, whereas an agent who always explores will never achieve the highest reward. A key component of RL is balancing exploration with exploitation to ensure the agent experiences a sufficient number of different actions, while still finding an optimal solution to the problem.

The agent in this problem uses an action-value estimate $Q(a)$ as a decision guide, supplying an expectation of future reward, based on historical actions taken. This action-value estimate $Q(a)$ is updated incrementally each time an action is taken. From this, the agent is able to make decisions based on what will achieve the highest expected reward.

The multi-armed bandit problem consists of a single discrete state space, meaning there are no transitions between states. The action space consists of $|A| = 10$ possible actions, one for each bandit arm. Each action in the action space is tied to a Gaussian distribution with differing centers, meaning each reward distribution will differ slightly, given a large enough sample size. When an action is selected, a reward is returned. This process is repeated for a certain number of steps making this experimental framework non-episodic. For this project we experiment with an ε-greedy policy and an upper confidence bound (UCB) policy, used for action space exploration.

It is expected that the UCB policies will outperform the ε-greedy policies, since it guarantees a balance of tried actions. In order for the model to successfully determine the most viable action, it has to try all possible actions. UCB ensures that the action exploration is balanced.

Section 2: Deliverables

GitHub Repository

`GitHub Repository: https://github.com/cooper-rm/multi-armed-bandit`

Implementation Summary

This lab consisted of two parts implemented across three Jupyter notebooks using PyTorch.

In Part 1, I built a custom Gaussian 10-arm bandit environment following the Gymnasium API structure with `reset()` and `step()` methods. I implemented two agent classes: an ε-greedy agent (tested with $\varepsilon \in \{0.01, 0.1, 0.2\}$) and a UCB agent (tested with $c \in \{1, 2\}$). Both agents use the incremental update rule to maintain action-value estimates. The experiment ran 1000 independent bandit problems, each for 2000 steps, with each agent facing the same true arm values per run but receiving independent reward noise.

In Part 2, I explored two standard Gymnasium environments: FrozenLake-v1 and Taxi-v3. I inspected their observation and action spaces and how the MDP tuple (S, A, R, P, γ) maps to the Gymnasium API. To facilitate better understanding I measured a baseline performance with a random policy agent over 1000 episodes each.

Key Results & Analysis

2.3.1 Part 1: Multi-Armed Bandit

The results from the 10-armed bandit experiment clearly demonstrate the exploration-exploitation tradeoff described in Sutton and Barto Chapter 2. This lab tested five variations, three for the ε-greedy policy and two for UCB.

Table **??** summarizes the key quantitative results.

Strategy	**Reward** (step 2000)	**% Optimal** (step 2000)	**Early Avg** (1–100)	**Late Avg** (1900–2000)
$\varepsilon = 0.01$	1.3734	69.10%	0.9329	1.3788
$\varepsilon = 0.1$	1.3581	82.30%	0.9776	1.3449
$\varepsilon = 0.2$	1.1829	74.60%	0.9323	1.1965
UCB $c = 1$	1.5000	94.70%	1.1664	1.5029
UCB $c = 2$	1.4736	89.80%	0.9327	1.4813

Table 1: Agent performance across 1000 runs of 2000 steps. Early/Late Avg is the mean reward over the first/last 100 steps respectively.

The best overall policy was UCB where c=1, achieving 94.7As expected, UCB outperformed the epsilon-greedy policy, as it was able to quickly balance action exploration and slowly decay exploration leading to higher exploitation at step 2000. Since epsilon-greedy explores randomly, it risks selecting some actions more often than others.

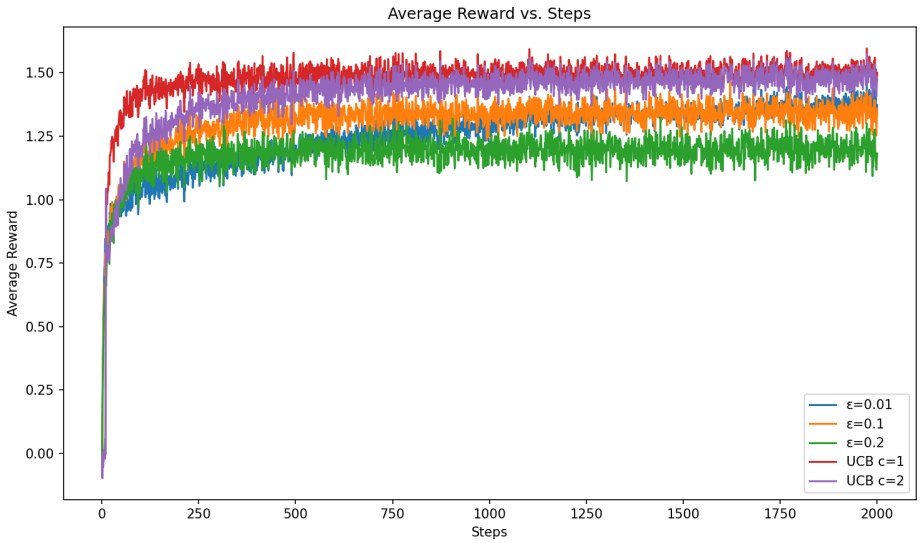

Figure 1: Average reward over 2000 steps across 1000 independent runs. UCB with $c = 1$ achieves the highest asymptotic reward (1.50), converging rapidly within the first 200 steps. The ε-greedy agents plateau at progressively lower levels as ε increases.

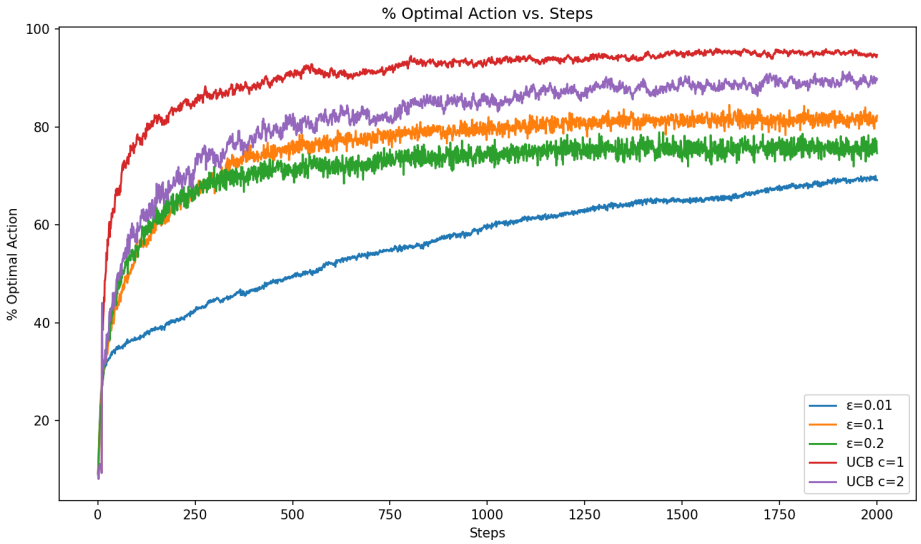

Figure 2: Percentage of runs selecting the optimal arm at each time step. UCB $c = 1$ reaches 94.70% by step 2000. Each ε-greedy agent is capped below 100% because it always explores at rate ε.

2.3.2 Part 2: Gymnasium Environments

Along with solving the multi-armed bandit problem I implemented two Gymnasium environments around Markov decision processes (MDP) including FrozenLake-v1 and Taxi-v3. For both these environments I implemented a random agent. Both of the random agents

were very unsuccessful at identifying the right actions to take, which was expected due to their randomness. FrozenLake-v1 consists of a binary reward and as expected resulted in around 1.5success rate was 0

Metric	**FrozenLake-v1**	**Taxi-v3**
States $\|S\|$	16 (4×4 grid)	500 (25 pos × 5 pass × 4 dest)
Actions $\|A\|$	4	6
Transitions	Stochastic	Deterministic
Reward type	Sparse ($\{0, +1\}$)	Shaped ($\{-10, -1, +20\}$)
Success rate	1.00%	0.00%
Avg reward / episode	0.010	-773.25
Avg episode length	8.0 steps	196.9 steps

Table 2: Environment comparison and random agent baseline performance over 1000 episodes.

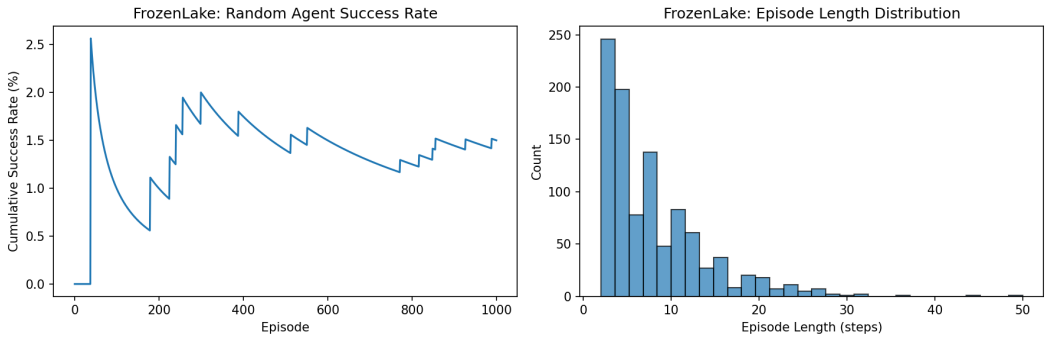

Figure 3: FrozenLake-v1 random agent: cumulative success rate over 1000 episodes (left) and episode length distribution (right).

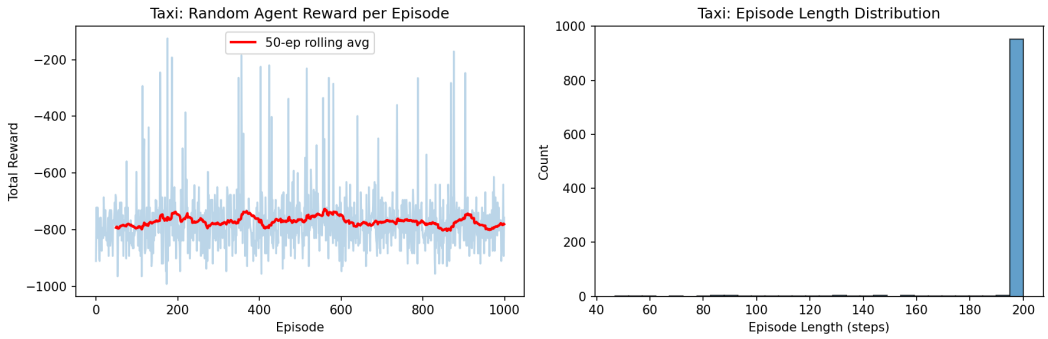

Figure 4: Taxi-v3 random agent: reward per episode with 50-episode rolling average (left) and episode length distribution (right).

Section 3: AI Use Reflection

Initial Interaction

Initially I asked Claude Code to help work through the multi-armed bandit problem step by step. I asked it to explain each aspect of the problem and explain how and why it implemented each code section. I additionally conversed with ChatGPT around different concepts like upper confidence bound (UCB) until I completely understood the material for the week.

Iteration Cycle

Iteration 1: At first all agents shared one bandit. The issue I had with this was clean separability. I decided to implement a single bandit per agent using the same seed. This resulted in the same true means and distributions but helped me break down and keep experiments separated as I worked through each area.

Iteration 2:

As I continued working through the problem I noticed there was a triple-nested loop which was concerning computationally. After adding logging every 100 runs I tracked the timing and decided not to break the structure on a small experiment like this. Claude Code suggested vectorization to reduce the complexity from approximately 10M to approximately 10K iterations, but I decided to keep the simple formatting since the runs didn't take long enough to require this level of complexity.

Iteration 3:

After reviewing the structure of things I was having a difficult time understanding why we used 1000 different bandits instead of one. However, after discussing that we run 1000 independent experiments with 2000 steps each, it makes sense to vary the seed for the different bandits. This helps with sampling bias and ensures the experiments are in the end a true representation of each parameter setting.

Critical Evaluation

When evaluating the results of the multi-armed bandits problem I verified they look roughly the same as the Sutton and Barto Figure 2.2 patterns. I also confirmed that my hypothesis of UCB outperforming epsilon-greedy policies was an accurate assumption. As far as Part 2 goes, it was interesting to see the different experimental structures but as expected, I witnessed random behavior from the random agents.

Learning Reflection

I found this project to be quite interesting. Overall, I discovered that exploration vs exploitation is fundamental to RL. Agents cannot learn without exploration, yet too much exploration and the agent won't perform well at all. Beyond just the theory is the data, which undeniably explains the value of well thought out exploration strategies. Furthermore,

even though Claude could have got the job done, without my questioning of its validity I would have learned substantially less. Next time I would further expand each idea before beginning to code.

Section 4: Speaker Notes

- **Problem:** The multi-armed bandit problem is fundamental to reinforcement learning.
- **Method:** Built epsilon-greedy and UCB agents in PyTorch, ran 1000 independent trials with 2000 steps each.
- **Design choice:** Each agent got its own bandit instance with the same seed, so they faced the same true environments but received independent rewards.
- **Key result:** UCB with $c = 1$ reached 94.7% optimal action selection with reward at 1.50 by step 2000.
- **Insight:** The epsilon-greedy agent hits a permanent ceiling because it never fully stops exploring, meaning no full exploitation.
- **Challenge:** The biggest challenge is managing the rate of exploration versus exploitation.
- **Connection:** The bandit is a single-state MDP with no transitions. The same incremental update rule used here extends directly to Q-learning in multi-state environments.